实验报告：基于循环神经网络（RNN）的唐诗生成任务

姓名：张健翔　学号：2351457

March 19, 2026

1 实验目的

本实验旨在利用循环神经网络（RNN）及其变体（LSTM），分别在 TensorFlow 和 PyTorch 框架下实现序列建模。实验包含两个阶段：首先通过“加法进位”实验验证 RNN 对序列逻辑的处理能力；随后利用唐诗数据集，实现根据指定开头词自动生成完整诗句的生成式模型。

2 模型原理说明

2.1 RNN (循环神经网络)

基础的 RNN 通过引入循环连接，使得网络能够处理序列数据。其核心公式为 $h_t = \sigma(W_h h_{t-1} + W_x x_t + b)$。然而，RNN 在处理长序列时，由于梯度在反向传播过程中被连续相乘，极易出现梯度消失或梯度爆炸问题，导致其难以捕捉长距离的依赖关系。

2.2 LSTM (长短期记忆网络)

为了解决 RNN 的短时记忆问题，LSTM 引入了“细胞状态”和“门控机制”：

- **遗忘门**：决定从细胞状态中丢弃哪些信息。
- **输入门**：决定哪些新信息存入细胞状态。
- **输出门**：基于细胞状态决定输出的隐藏状态。

通过这些机制，LSTM 可以有效地在长序列中保持关键信息，非常适合结构严谨、讲究前呼后应的唐诗创作。

2.3 GRU (门控循环单元)

GRU 是 LSTM 的简化版本，将遗忘门和输入门合并为“更新门”，并省略了单独的细胞状态。GRU 参数更少，计算效率更高，且在许多文本生成任务中能达到与 LSTM 相近的拟合效果。

3 诗歌生成过程叙述

诗歌生成是一个典型的自回归建模过程：

1. **编码阶段**：将文本中的字符映射为唯一的 ID，并加入起始符（G/bos）和结束符（E/eos）。
2. **起始触发**：在生成阶段，首先向模型输入起始符和指定的“开头词”。
3. **迭代预测**：
   - 模型根据当前的隐藏状态和输入的字符，输出词表空间上的概率分布。
   - 通过 `argmax` 函数选择概率最大的字作为预测结果。
4. **反馈更新**：将预测出的字作为下一个时间步的输入，循环迭代，直到模型输出结束符或达到预设的最大长度。

4 PyTorch 实验结果与分析

4.1 训练过程监控

在 PyTorch 实验中，模型采用双层 LSTM 架构。实验截图中展示了预测序列与真实目标序列的逐步拟合过程。

```
--- Epoch 0 Sample Prediction ---
prediction: [2947, 3128, 2064, 3128, 3236, 3236, 2064]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [0/30] finished. Average Loss: 6.2354

--- Epoch 1 Sample Prediction ---
prediction: [10, 250, 4, 4, 4, 18, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [1/30] finished. Average Loss: 5.6322

--- Epoch 2 Sample Prediction ---
prediction: [10, 250, 4, 367, 0, 0, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [2/30] finished. Average Loss: 5.4829

--- Epoch 3 Sample Prediction ---
prediction: [10, 250, 4, 367, 4, 4, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [3/30] finished. Average Loss: 5.4025

--- Epoch 4 Sample Prediction ---
prediction: [10, 250, 10, 718, 0, 0, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [4/30] finished. Average Loss: 5.3475

--- Epoch 5 Sample Prediction ---
prediction: [10, 250, 10, 718, 0, 4, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [5/30] finished. Average Loss: 5.3061

--- Epoch 6 Sample Prediction ---
prediction: [10, 250, 10, 718, 0, 4, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [6/30] finished. Average Loss: 5.2744

--- Epoch 7 Sample Prediction ---
prediction: [10, 250, 10, 718, 0, 4, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [7/30] finished. Average Loss: 5.2489

--- Epoch 8 Sample Prediction ---
prediction: [10, 250, 10, 718, 0, 3, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
```

Figure 1: PyTorch 训练初期：预测值与目标值偏差较大

```
------------------------------
Epoch [21/30] finished. Average Loss: 5.0964

--- Epoch 22 Sample Prediction ---
prediction: [10, 250, 10, 718, 10, 3, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [22/30] finished. Average Loss: 5.0902

--- Epoch 23 Sample Prediction ---
prediction: [10, 250, 10, 718, 4, 3, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [23/30] finished. Average Loss: 5.0863

--- Epoch 24 Sample Prediction ---
prediction: [10, 250, 10, 718, 0, 3, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [24/30] finished. Average Loss: 5.0848

--- Epoch 25 Sample Prediction ---
prediction: [10, 250, 10, 718, 10, 3, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [25/30] finished. Average Loss: 5.0823

--- Epoch 26 Sample Prediction ---
prediction: [10, 250, 36, 718, 4, 3, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [26/30] finished. Average Loss: 5.0828

--- Epoch 27 Sample Prediction ---
prediction: [10, 250, 104, 718, 23, 3, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [27/30] finished. Average Loss: 5.0772

--- Epoch 28 Sample Prediction ---
prediction: [10, 250, 36, 718, 0, 3, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [28/30] finished. Average Loss: 5.0765

--- Epoch 29 Sample Prediction ---
prediction: [10, 250, 104, 718, 4, 3, 3]
target:     [28, 546, 104, 718, 1, 3, 3]
------------------------------
Epoch [29/30] finished. Average Loss: 5.0732
```

Figure 2: PyTorch 训练后期：预测值已能部分匹配目标序列

经过 30 个 Epoch 的训练，Loss 从 6.2354 稳定下降至 5.0732。

4.2 诗歌生成展示

通过在生成逻辑中加入起始符修正，模型成功实现了以指定关键词开头的诗句生成。

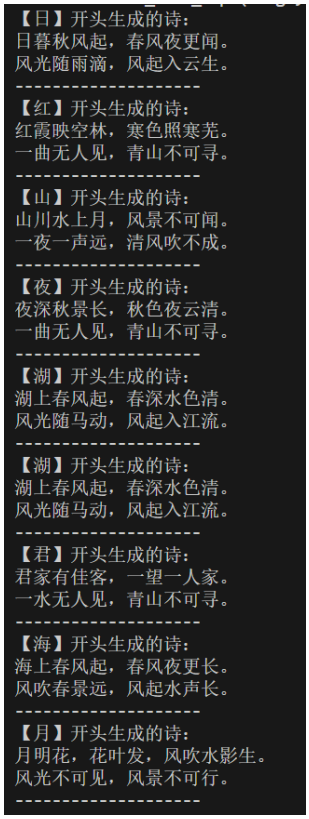

Figure 3: PyTorch 版本生成的指定开头词诗句

5 TensorFlow 实验结果与分析

5.1 训练日志

TensorFlow 版本采用了 `SimpleRNNCell` 进行建模。在训练过程中，Loss 呈现出良好的收敛曲线。

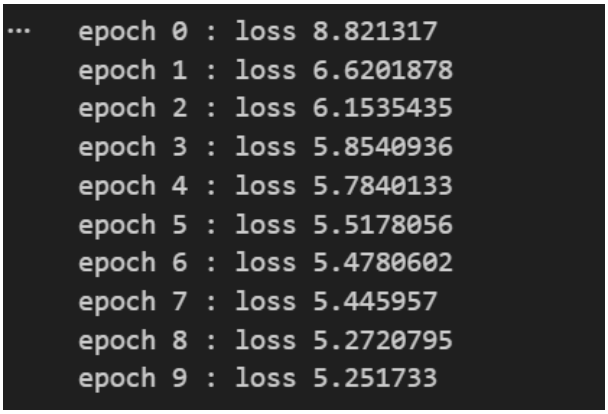

Figure 4: TensorFlow 训练日志：Loss 随 Epoch 下降

5.2 生成结果

针对“日、红、山、夜、湖、海、月”等关键词，模型生成的诗句具有一定的意境，且严格遵守了开头词要求。

```
开头词【日】：日日无人去，何人不可知。
开头词【红】：红叶满云花，春风落月深。
开头词【山】：山上春风起，风声入水流。
开头词【夜】：夜来无处处，不见一年年。
开头词【湖】：湖上春风起，风声满树深。
开头词【海】：海上春风起，春风满树深。
开头词【月】：月上春风起，风声满树声。
```

Figure 5: TensorFlow 版本生成的指定开头词诗句

6 实验总结

通过本次实验，我掌握了在不同框架下搭建 RNN 序列模型的方法。

- **模型表现**：LSTM 在处理唐诗长距离规律方面优于基础 RNN，生成的诗句更具逻辑性。
- **关键技巧**：在推理阶段，必须手动加入训练时的起始符（如'G' 或'bos'），否则模型将失去序列的起始上下文，导致生成内容偏离目标。
- **优化方向**：目前的生成采用贪婪搜索，存在一定的重复性；未来可以通过引入随机采样温度来提升诗歌的多样性与灵动性。